

Strategic Optimization of Deep Learning Pipelines for Bone Metastasis Detection: A Forensic Analysis of the BS-80K Scintigraphy Ecosystem

The clinical imperative for the automated screening of skeletal metastasis is deeply rooted in the high morbidity and mortality associated with osseous dissemination from primary solid tumors, most notably those originating in the breast, prostate, and lungs.¹ Bone scintigraphy remains the global screening standard for these conditions, utilizing the radiopharmaceutical technetium-99m labeled methylene diphosphonate (^{99m}Tc -MDP).¹ The mechanism of this tracer involves chemisorption onto hydroxyapatite crystals at sites characterized by elevated skeletal vascularity and bone turnover.¹ This physiological process allows for the detection of metabolic changes that often precede structural evidence visible on computed tomography (CT) or conventional radiography by several months.¹ Despite its profound sensitivity, bone scintigraphy is frequently compromised by low specificity, as focal areas of tracer hyperaccumulation—colloquially termed "hot spots"—can be produced by a variety of benign etiologies, including degenerative osteoarthritis, trauma, old fractures, Paget's disease, and physiological excretion in the bladder.¹ The integration of deep learning (DL) into this diagnostic pathway offers the potential to transcend the subjective limitations of manual interpretation, which is often hindered by high inter-rater variability and significant clinician workload.¹

The release of the BS-80K dataset, a massive repository of 82,544 images derived from 3,247 patients at West China Hospital, has fundamentally altered the landscape of nuclear medicine research.¹ However, a rigorous forensic audit of the emerging literature reveals a significant disconnect between reported academic benchmark metrics—often exceeding 95% or 97% accuracy—and genuine clinical utility.¹ This discrepancy arises from systemic logic errors inherent in standard image-level processing pipelines, most notably data leakage and the catastrophic loss of spatial information during image resizing.¹ To develop a pipeline that is both high-performing and clinically robust, it is necessary to move beyond treating bone scans as natural images and instead treat the problem as a radiological reasoning task.¹ This requires a specialized architectural blueprint that preserves spatial resolution, correlates multi-view perspectives, and addresses extreme class imbalance through advanced loss function engineering and a structured Sim2Real curriculum.¹

Structural Anatomy of the BS-80K Dataset and the

Logic of Data Independence

The foundational substrate of any deep learning pipeline is the dataset itself, yet the BS-80K dataset is often fundamentally misunderstood as a "bag of images" rather than a hierarchical collection of patient studies.¹ Each unique patient study generates a primary Anterior (ANT) and Posterior (POST) whole-body projection.¹ From these primary views, the dataset creators extracted 13 anatomical regional slices and specific bounding box crops around suspicious hot spots, leading to a total image count of over 82,000.¹ This structure creates a complex "dependency graph" where dozens of images are linked by a single biological identity, sharing identical skeletal age, pathological burden, and scanner-specific noise signatures.¹

The most pervasive logic error identified in existing research involve the treatment of these correlated images as independent samples during the dataset splitting process.¹ When researchers employ a standard 80/20 split based on the total image count, the resulting validation and test sets are contaminated with "leaked" information.¹ For example, if a patient's anterior view is in the training set and their posterior view or a regional spinal crop is in the test set, the model does not learn the general semantic features of metastasis.¹ Instead, it learns to recognize the specific, unique anatomical markers of that individual patient, such as a particular degree of scoliosis or a benign degenerative pattern in the knee.¹ Consequently, reported accuracies of 97% are likely artifacts of memorization rather than indicators of diagnostic generalization.¹ A truly honest baseline, enforced by patient-level splitting, typically yields performance between 83% and 88% accuracy.¹

Dependency Level	Source Count	Biological Correlation	Validation Risk	Implementati on Protocol
Patient Level	3,247	None (Independent)	Gold Standard (Safe)	GroupShuffleS plit ¹
Study/Session Level	~3,247	High (Within-sessio n)	Minimal (Unless longitudinal)	GroupKFold ¹
View Level (ANT/POST)	~6,500	Extremely High	Critical Leakage Risk	Siamese Pairing ¹
Regional Level	~82,000+	Absolute	Catastrophic Leakage Risk	Strict Grouping ¹

To rectify this, the pipeline must implement a strict patient-level split using libraries such as

scikit-learn's GroupShuffleSplit or GroupKFold.¹ This ensures that any given patient's data is exclusively contained within one split, forcing the network to learn the universal features of disease rather than individual anatomical signatures.³ This rigorous approach is the only way to ensure that the target of >90% accuracy is legitimate and can be delivered with the transparency required for real-world oncological screening.¹

The Resolution Bottleneck and Spatial Fidelity in Whole-Body Scintigraphy

A critical decision in many current pipelines is the exclusive use of full-body scans while opting out of regional crops.¹ While this simplifies data management, it introduces a severe informational bottleneck during the image resizing phase.¹ A standard whole-body bone scan is a non-square, high-aspect-ratio image, typically 1024 pixels in length and 256 pixels in width.¹ Standard convolutional backbones, such as ResNet, EfficientNet, or MobileNetV2, generally expect square inputs, typically 224x224 or 256x256 pixels.¹

The current standard strategy involving square padding followed by resizing to 256x256 pixels results in a highly sparse image where approximately 75% of the pixel area is dedicated to black padding.¹ When the original 1024-pixel height is compressed into a 256-pixel frame, a catastrophic loss of spatial detail occurs.¹ A small metastatic lesion, which might occupy only 5x5 pixels in the original scan, may be reduced to a sub-pixel blur that is indistinguishable from background noise or sensor artifacts.¹ Research into medical diagnostic tasks indicates that while some labels (like cardiomegaly) plateau at lower resolutions, focal pathologies such as nodules or metastases require significantly higher spatial fidelity.¹

To maintain diagnostic performance while using only whole-body scans, the pipeline architecture must be flexible.¹ For starters, testing using pretrained models with image resizing requires a cell where img_resize is assigned a variable name, allowing for the simple alteration of resize and batch size parameters.¹ This allows the researcher to decide if the pipeline will work effectively if trained from scratch by observing how performance scales with resolution.¹ A superior architectural logic for achieving >90% accuracy involves either accommodating higher input dimensions (e.g., 672x224 or 1024x256) or utilizing "Tandem Architectures" that process the whole body at low resolution for global context while simultaneously analyzing high-resolution local patches or tiles.¹

Processing Strategy	Resolution Impact	Computational Cost	Spatial Integrity	Clinical Suitability
Naive Resize	High	Low	Distorted	Unsuitable ¹

(256x256)	Degradation			
Square Padding (256x256)	Extreme Degradation	Low	25% Active Area	Poor ¹
High-Dim Input (1024x256)	None	High	100% Active Area	Optimal ¹
Overlapping Tiling	None	Moderate	100% Active Area	Optimal ¹

The decision to opt out of regional crops is identified as a "missed opportunity" for feature pre-training, as these crops provide the high-resolution textural examples the global model currently lacks.¹ Integrating these crops into the training phase allows the model to learn local feature representations before tackling the global skeletal pattern.¹

Preprocessing Integrity and Intensity Normalization Logic

Deep learning models are highly sensitive to input distributions, and scintigraphy pixel values are notoriously arbitrary.¹ Unlike natural images where values are constrained between 0 and 255, scintigraphy counts range from zero to thousands depending on metabolic state and scan duration.¹ The presence of "hot" regions, such as the bladder (due to renal excretion) or injection site artifacts, creates extreme outliers that skew standard min-max normalization.¹

A robust implementation should utilize 99.9% percentile clipping to effectively neutralize these artifacts, preventing them from compressing the diagnostic contrast of the skeleton.¹ Following clipping, Contrast Limited Adaptive Histogram Equalization (CLAHE) is mandatory for enhancing local contrast, making subtle "hot spots" in the ribs or spine more distinct against the background.¹ Evidence suggests that CLAHE can improve classification precision by nearly 10% when combined with data augmentation.¹ However, a significant "preprocessing void" remains in many pipelines: the lack of automated bladder-masking or segmentation-guided normalization.¹ Extremely high radiation in non-lesion areas can lead to the "big numbers eat small numbers" phenomenon, where the model's global pooling layers prioritize the intense bladder signal over smaller, clinically significant metastatic lesions.¹

The implementation of cv2.INTER_AREA for resizing is a mathematically sound choice for downsampling, as it averages pixel values within a neighborhood, maintaining better texture

integrity than bilinear or bicubic interpolation.¹ However, no interpolation method can compensate for the initial 4x spatial compression.¹ A superior logic would involve a multi-scale approach where CLAHE is applied to the full-resolution image before any cropping or resizing, ensuring textural details are enhanced before being subjected to spatial degradation.¹

Architectural Optimization: From Pretrained Backbones to Siamese Fusion

The current pipeline exploration involves testing pretrained models such as MobileNetV2, ResNet50, and VGG16 with image resizing.¹ These models, typically trained on ImageNet, provide a strong feature extraction foundation but require specific adaptations for nuclear medicine.¹ The use of the Keras Model API allows for these backbones to be frozen during initial training, focusing on the custom top layers, which typically include GlobalAveragePooling2D, Flatten, and Dense layers with ReLU and Dropout for regularization.¹

However, the most significant architectural flaw in current pipelines is the treatment of Anterior and Posterior views as independent samples.¹ In clinical practice, a radiologist never interprets an Anterior view in isolation.¹ A lesion on the spine will be closest to the detector in the Posterior view, presenting as a sharp, intense area of uptake.¹ In the Anterior view, however, the same lesion is attenuated by the thickness of the body and obscured by the sternum, appearing faint or invisible.¹ A model processing only the Anterior view would correctly call it "Normal," while the patient is actually "Positive," resulting in a high False Negative rate that compromises clinical safety.¹

To achieve accuracy >90%, the architecture must employ Multi-View Fusion.¹ This involves a dual-stream or Siamese network where two identical backbones (sharing weights) process the ANT and POST views simultaneously.¹ Sharing weights is critical because the synthetic data often used to supplement training is frequently single-view; weight sharing allows the features learned from synthetic images to be automatically applied to both real-world views.¹ The feature vectors from both branches are then fused—via concatenation, element-wise addition, or cross-attention—before the final classification head.¹

Backbone Model	Parameters	Advantage	Disadvantage	Suitability
MobileNetV2	~3.4M	Lightweight, fast	Limited capacity for noise	Baseline/Mobile ¹

ResNet50	~25.6M	Skip connections aid gradients	High memory usage	High ¹
VGG16	~138M	Simple architecture	Prone to overfitting	Moderate ¹
EfficientNet-B4	~19M	Scalable, efficient	Complex structure	Optimal ¹

Standard CNN backbones utilize Global Average Pooling (GAP) to condense feature maps, but in bone scans, this is a logic error.¹ A metastasis is a focal pathology occupying less than 1% of the image area.¹ Averaging features over the entire skeleton dilutes the metastatic signal with the noise of healthy bone.¹ The integration of Attention Mechanisms, such as Triplet Attention or the Convolutional Block Attention Module (CBAM), is crucial for achieving >90% accuracy, as these modules learn to generate a heatmap weighting informative regions more heavily.¹

Managing Heavy Class Imbalance: Loss Function Engineering and Sampling

Real-world oncology screening is inherently an imbalanced problem; in the BS-80K ecosystem, the prevalence of metastasis is approximately 30%, though this varies by primary cancer type and region.¹ In the specific Paraguayan breast cancer dataset, only 21% of patients (60 out of 291) were diagnosed with metastases.¹ This 70/30 or 80/20 imbalance renders standard Accuracy a deceptive metric; a "Zero-Rule" classifier predicting "Negative" for every image would achieve 70-80% accuracy while failing clinical requirements.¹

The cost of a False Negative (missing a cancer diagnosis) is orders of magnitude higher than a False Positive.¹ Standard Cross-Entropy (CE) fails because it does not account for this disparity.¹ The current pipeline must be optimized for Sensitivity (Recall) using Weighted Focal Loss.¹ Focal Loss modifies standard CE by adding a modulating factor $(1 - p_t)^\gamma$ that down-weights "easy" samples (clear normal scans) and forces the model to focus on "hard" samples like faint metastases or complex degenerative arthritis.¹

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t)$$

1

The focusing parameter (gamma) and the balancing parameter (alpha) allow for the

fine-tuning of the model's sensitivity.¹ By setting alpha to favor the positive class, the pipeline can be optimized for high recall—a primary clinical requirement for screening tools.¹ Furthermore, the pipeline should integrate Hard Negative Mining.¹ After each epoch, the system identifies "Normal" patients (e.g., those with severe arthritis) misclassified as "Metastasis" and oversamples these cases in the next epoch, forcing the model to learn the subtle differences between symmetric, joint-focused uptake and the asymmetric, random distribution of malignancy.¹

Loss Function	Imbalance Handling	Gradient Focus	Clinical Priority	Implementation
Cross-Entropy	None	Uniform	Accuracy	BinaryCrossentropy ²⁶
Weighted CE	High	Uniform	Recall	pos_weight ²⁰
Focal Loss	Very High	Hard Examples	Recall/Precision	BinaryFocalCrossentropy ²⁶
Dice/Tversky Loss	Extreme	Overlap-focused	Segmental Sensitivity	Custom ²⁹

To implement this dynamically, the compute_class_weight function from sklearn can be used to calculate weights based on the inverse class frequency, which are then passed to the model.fit() method via the class_weight dictionary.¹⁹ This ensures that the model pay more attention to the minority (metastasis) class during training.³⁰

The Synthetic Data Paradigm: Bridging the Sim2Real Gap

The inclusion of synthetic data is a strategic necessity to address class imbalance and the "long-tail" of rare metastatic presentations.¹ While StyleGAN2 or Diffusion models can generate diverse pathological patterns, the traditional mixing of real and synthetic data in a single training phase can lead to "synthetic bias".¹ Synthetic images often lack the specific "Poisson noise" and sensor artifacts characteristic of real gamma camera acquisitions; if a model learns to detect "smooth" synthetic lesions, it may fail to identify "noisy" real lesions in a clinical setting.¹

Research in Sim2Real transfer (Simulation to Reality) suggests that the optimal path involves a structured "Curriculum Learning" or "Two-Stage Fine-Tuning" strategy.¹ This approach

prevents "Catastrophic Forgetting" of real features.¹

- 1. **Phase 1 (Synthetic Foundation):** Train the Siamese backbone exclusively on high-volume synthetic data for 30-50 epochs with a focus on "Hot Spot Detection".¹
- 2. **Phase 2 (Real-World Refinement):** Freeze the early feature extraction layers and fine-tune on high-fidelity real BS-80K data and local clinical data.¹

This staged approach, sometimes called "bridged transfer," has demonstrated significantly higher accuracy than simple mixing, as it prevents the subtle, high-fidelity signals present in real scans from being washed out by synthetic samples.¹

Parameter-Driven Experimental Setup: Variable Resize and Batch Size

The user's original request emphasizes the need for a pipeline that is easily configurable for testing.¹ Specifically, a cell should be created where image resize and batch size are assigned variable names to allow for rapid iteration.¹ This is crucial for deciding whether the pipeline will perform adequately if trained from scratch, as it allows for the empirical observation of how computational resources (batch size) and spatial resolution (resize) impact convergence and diagnostic fidelity.¹

Parameter	Variable Name	Range (Suggested)	Impact on Convergence	Impact on Quality
Image Height	IMG_HEIGHT	224 to 1024	Memory constraints	Focal detail preservation ¹
Image Width	IMG_WIDTH	224 to 256	Memory constraints	Aspect ratio integrity ¹
Batch Size	BATCH_SIZE	16 to 128	Gradient stability	Training throughput ¹⁰
Resize Method	INTERP_METHOD	INTER_AREA/LI NEAR	Computational cost	Texture preservation ¹

This parameterization allows the pipeline to scale from low-resolution testing using pretrained backbones to high-resolution production training from scratch.¹ Furthermore, using the tf.data.Dataset API or ImageDataGenerator with these parameters enables the real-time processing of images, ensuring that data augmentation—such as rotation, flipping, and

shearing—can be applied on the fly to further mitigate the risk of overfitting.¹⁰

Validation Rigor and Clinical Success Metrics

The forensic audit confirms that reported accuracies of 97% are likely artifacts of data leakage, whereas honest, patient-level splits yield metrics near 83%.¹ To bridge this gap and reach the >90% target, the cumulative gains from architectural realignment must be realized.¹

- **Resolution Gains:** Solving the resolution bottleneck through tiling or high-dim backbones typically yields a 5-7% gain in sensitivity.¹
- **Fusion Gains:** Implementing multi-view Siamese fusion resolves ambiguous 2D projections, adding 4-6% to overall accuracy.¹
- **Curriculum Gains:** Adopting a Sim2Real curriculum stabilizes feature learning, contributing another 3-5%.¹

In this domain, accuracy alone is insufficient; the clinical requirement is a system with high Sensitivity (to not miss cancer) and manageable Specificity (to not over-call arthritis).¹ The F1-score—the harmonic mean of Precision and Recall—is the superior metric for this domain.¹ A model with a Projected F1-score of >0.91 represents a significant improvement over the current honest baseline of 0.86.¹

Strategy	Accuracy	Sensitivity	Specificity	F1-Score	Validation Reliability
Naive Image-Split (Leaked)	97.4%	95.8%	-	-	Invalid ¹
Honest Single-View (Baseline)	83.2%	79.3%	90.7%	0.86	High ¹
mSegResRF (Regional)	86.8%	85.0%	-	0.86	High ¹
Proposed Fusion-Curriculum	>92.0%	>94.0%	>91.0%	>0.91	Optimal ¹

Conclusion: A Strategic Blueprint for Clinical

Diagnostic Excellence

The path toward achieving greater than 90% accuracy in bone metastasis classification is not found through black-box natural image classification but through a "Gray Box" approach that respects the radiological physics and clinical logic of the BS-80K ecosystem.¹ The forensic audit demonstrates that the highest reported metrics in the literature are often illusory, born of data leakage logic errors where patient identity is not isolated.¹ To achieve genuine clinical utility, researchers must enforce strict patient-level splitting using protocols like GroupShuffleSplit.¹

The pipeline must address the catastrophic resolution collapse caused by compressing high-aspect-ratio scintigrams into standard square frames.¹ By implementing a parameter-driven system that allows for variable resizing and batch sizing, researchers can evaluate the trade-offs between computational throughput and diagnostic fidelity.¹ Architectural realignment must move toward Siamese Multi-View Fusion, mimicking the radiologist's cognitive process of correlating Anterior and Posterior views.¹

Furthermore, extreme class imbalance must be managed through the integration of Weighted Focal Loss and active Hard Negative Mining, which force the model to focus on challenging benign confounders like arthritis.¹ Finally, the use of a Sim2Real curriculum ensures that synthetic data acts as a sophisticated pre-training tool rather than a source of bias.¹ By following this strategic blueprint, the target of >90% clinically valid accuracy is not only achievable but can be delivered with the robustness, transparency, and reliability required for the high-stakes diagnostic environment of modern oncology.¹

Works cited

1. Critical Analysis of Deep Learning Pipelines for Bone Metastasis Classification.docx
2. A robust model training strategy using hard negative mining in a weakly labeled dataset for lymphatic invasion in gastric cancer - NIH, accessed January 31, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC10766063/>
3. GroupKFold and Stratified GroupKFold Efficientnet - Kaggle, accessed January 31, 2026, <https://www.kaggle.com/code/reighns/groupkfold-and-stratified-groupkfold-efficientnet>
4. How to apply CLAHE on RGB color images? - Stack Overflow, accessed January 31, 2026, <https://stackoverflow.com/questions/25008458/how-to-apply-clahe-on-rgb-color-images>
5. GroupKFold — scikit-learn 1.8.0 documentation, accessed January 31, 2026, https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GroupKFold.html

6. Siamese network with Keras, TensorFlow, and Deep Learning - PyImageSearch, accessed January 31, 2026, <https://pyimagesearch.com/2020/11/30/siamese-networks-with-keras-tensorflow-and-deep-learning/>
7. How to Generate a Train-Test-Split Based on a Group ID? - GeeksforGeeks, accessed January 31, 2026, <https://www.geeksforgeeks.org/machine-learning/how-to-generate-a-train-test-split-based-on-a-group-id/>
8. Deep Learning and Medical Image Analysis with Keras - PyImageSearch, accessed January 31, 2026, <https://pyimagesearch.com/2018/12/03/deep-learning-and-medical-image-analysis-with-keras/>
9. Advanced Image Preprocessing with CLAHE - Kaggle, accessed January 31, 2026, <https://www.kaggle.com/code/kyleberdy/advanced-image-preprocessing-with-clahe>
10. Image Data Generators in Keras - Towards Data Science, accessed January 31, 2026, <https://towardsdatascience.com/image-data-generators-in-keras-7c5fc6928400/>
11. Load and preprocess images | TensorFlow Core, accessed January 31, 2026, https://www.tensorflow.org/tutorials/load_data/images
12. Tiling-images-with-Tensorflow - Kaggle, accessed January 31, 2026, <https://www.kaggle.com/code/gabrielchehade/tiling-images-with-tensorflow>
13. CLAHE Histogram Equalization - OpenCV - GeeksforGeeks, accessed January 31, 2026, <https://www.geeksforgeeks.org/python/clahe-histogram-equalization-opencv/>
14. Coding-Monarch/Retina_Preprocessing: Retina Preprocessing with CLAHE (Contrast limited adaptive histogram equalization) - GitHub, accessed January 31, 2026, https://github.com/Coding-Monarch/Retina_Preprocessing
15. Building powerful image classification models using very little data - The Keras Blog, accessed January 31, 2026, <https://blog.keras.io/building-powerful-image-classification-models-using-very-little-data.html>
16. Building Siamese Networks using TensorFlow's Functional API: A Comprehensive Guide, accessed January 31, 2026, <https://thegrigorian.medium.com/building-siamese-networks-using-tensorflows-functional-api-a-comprehensive-guide-a151a76ea9f0>
17. Implementation of a Siamese Network in Keras and TensorFlow - Regenerative, accessed January 31, 2026, <https://regenerativetoday.com/implementation-of-a-siamese-network-in-keras-and-tensorflow/>
18. A Friendly 😊 Introduction to Siamese Networks - Kaggle, accessed January 31, 2026, <https://www.kaggle.com/code/shindeshubham85/a-friendly-introduction-to-siamese-networks>

19. compute_class_weight — scikit-learn 1.8.0 documentation, accessed January 31, 2026,
https://scikit-learn.org/stable/modules/generated/sklearn.utils.class_weight.compute_class_weight.html
20. Customizing Loss Functions in PyTorch for Extremely Imbalanced Predictions: Complete With Healthcare Code Example - Jesse Villines, accessed January 31, 2026,
https://jesservillines.medium.com/customizing-loss-functions-in-pytorch-for-extremely-imbalanced-predictions-complete-with-ecc9c915f3b6?source=rss-----a-rtificial_intelligence-5
21. Use weighted loss function to solve imbalanced data classification problems - Medium, accessed January 31, 2026,
<https://medium.com/@zergtant/use-weighted-loss-function-to-solve-imbalanced-d-data-classification-problems-749237f38b75>
22. Computes focal cross-entropy loss between true labels and predictions. — loss_binary_focal_crossentropy - keras3, accessed January 31, 2026,
https://keras3.posit.co/reference/loss_binary_focal_crossentropy.html
23. focal_loss.BinaryFocalLoss — focal-loss 0.0.8 documentation, accessed January 31, 2026,
https://focal-loss.readthedocs.io/en/latest/generated/focal_loss.BinaryFocalLoss.html
24. sigmoid_focal_loss — Torchvision main documentation, accessed January 31, 2026,
https://docs.pytorch.org/vision/main/generated/torchvision.ops.sigmoid_focal_loss.html
25. focal_loss.binary_focal_loss — focal-loss 0.0.8 documentation, accessed January 31, 2026,
https://focal-loss.readthedocs.io/en/latest/generated/focal_loss.binary_focal_loss.html
26. Probabilistic losses - Keras, accessed January 31, 2026,
https://keras.io/api/losses/probabilistic_losses/
27. Hard Negative Sample Mining for Whole Slide Image Classification - PMC, accessed January 31, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12185924/>
28. HARD NEGATIVE MINING. A SIMPLE EXPLANATION ABOUT HARD... | by Naga sundar chokkakula | Medium, accessed January 31, 2026,
<https://medium.com/@sundardell955/hard-negative-mining-91b5792259c5>
29. Loss Function Library - Keras & PyTorch - Kaggle, accessed January 31, 2026,
<https://www.kaggle.com/code/bigironsphere/loss-function-library-keras-pytorch>
30. How Does the class_weight Parameter in Scikit-Learn Work? - GeeksforGeeks, accessed January 31, 2026,
<https://www.geeksforgeeks.org/machine-learning/how-does-the-classweight-parameter-in-scikit-learn-work/>
31. How to set class weights for imbalanced classes in Keras? - Data Science Stack Exchange, accessed January 31, 2026,
<https://datascience.stackexchange.com/questions/13490/how-to-set-class-weig>

[hts-for-imbalanced-classes-in-keras](#)

32. sklearn utils compute_class_weight function for large dataset - Stack Overflow, accessed January 31, 2026,
<https://stackoverflow.com/questions/60408901/sklearn-utils-compute-class-weight-function-for-large-dataset>
33. FocalLoss for Keras - Kaggle, accessed January 31, 2026,
<https://www.kaggle.com/code/rejpalcz/focalloss-for-keras/comments>
34. A detailed example of data generators with Keras, accessed January 31, 2026,
<https://stanford.edu/~shervine/blog/keras-how-to-generate-data-on-the-fly>
35. tf.data: Build TensorFlow input pipelines, accessed January 31, 2026,
<https://www.tensorflow.org/guide/data>